# EasyCrypt tricks

Every listing below is a complete file from this folder. Each compiles on its own against the standard library (checked with EasyCrypt release `r2026.10`), so any of them can be copied out, run, and changed. Sections 9–12 come from the Simon separation development; `TACTICS.md` in the same folder is a reference table of the less common tactics and their syntax.

```
for f in *.ec; do easycrypt compile "$f" || echo "FAILED: $f"; done
```

## Contents


1 Rules of thumb 1

2 Reading the rules 2

3 `conseq` 2

4 `call` 4

5 `exlim` and `seq` 6

6 Probabilities 8

7 Up to bad 9

8 Dropping one-sided calls 12

9 Expected values: `ehoare` 14

10 Samplings 18

11 Program transformations 21

12 From a random parameter to a fixed one 24

13 Pitfalls 25

14 Seeing the goals from the command line 25


## 1 Rules of thumb

- `exlim` **binds the value at the start of the remaining program.** That is the value in the current precondition. If the program changes it before the point where you need it, cut the program first with `seq` or `sp`. `exlim e ⇒ v` is the same as `exists* e; elim* ⇒ v`.
- **Calls are processed from the end.** When dropping one-sided calls, handle the last one first.

- **An equiv cannot relate a side to its own past.** Fix the old value with `exlim` and add a hoare triple for that side with `conseq E H1 H2`.
- `byequiv lemma` **needs an exact match.** Write the spec you want in `byequiv (: P ==> Q)` and then `conseq lemma`. If the games are on the other sides, use `symmetry` first.
- **An up-to-bad invariant must include the equality of the flag**, or nothing says the left flag is still down while the right one is.
- **A fact stated as** $\Pr[\dots] = 1$ **becomes a spec with** `bypr`**.** For a phoare spec, `bypr => &m <-; exact (h &m)` is enough. For a hoare spec, go through `Pr[mu_not]` as in Section 6.

## 2 Reading the rules

Each section below states, as inference rules, the program-logic rule behind the tactics it shows. The rules are simplified to the shape the tactic uses, and they are read bottom-up, because the tactic turns the goal below the line into the goals above it.

- $\models c_1 \sim c_2 : \Phi \Rightarrow \Psi$ is a pRHL judgement. From any two memories related by $\Phi$, the output distributions of $c_1$ and $c_2$ are related by $\Psi$, which means there is a coupling of them whose support lies in $\Psi$.
- $\Phi$ and $\Psi$ relate two memories. For a formula $P$ over one memory, $P\{1\}$ reads it in the left memory and $P\{2\}$ in the right one, and $=\{x\}$ abbreviates $x\{1\} = x\{2\}$.
- $\models c : P \Rightarrow Q$ is a Hoare judgement, saying that every terminating run of $c$ from a memory in $P$ ends in $Q$. It says nothing about termination.
- $\models c : P \Rightarrow Q\ [=p]$ is a probabilistic Hoare judgement, saying that from every memory in $P$ the probability that $c$ ends in $Q$ is exactly $p$. With $p = 1$ it gives both termination and the postcondition.
- $\models_{\mathbb{E}} c : f \Rightarrow g$ is an expectation Hoare judgement: $f$ and $g$ map a memory to a non-negative real or $\infty$, and from every memory $m$ the expectation of $g$ over the final memories is at most $f(m)$. A run that does not terminate contributes nothing.
- $\mathrm{mod}(c)$ is the set of variables $c$ may write. When a rule quantifies over values for them, EasyCrypt shows the quantified names as `result_L`, `O_R`, `S_R` and so on.

## 3 `conseq`

- `conseq H` applies a spec lemma and leaves two side goals, the precondition implication and the postcondition implication.
- `conseq (: _ ==> Q)` changes the postcondition to `Q` and keeps the precondition. Use it to strengthen the postcondition to the equalities that `sim` proves.
- `conseq E H1 H2` conjoins a hoare triple for each side onto a two-sided equiv. An equiv relates the two sides to each other, never a side to its own initial value, so facts like "the left counter went up by one" have to come from a one-sided hoare triple.
- `(_: true ==> true)` in place of `H1` or `H2` means that nothing is added on that side. It becomes a trivial goal.
- `conseq ll H` turns losslessness and a hoare triple into a phoare spec with probability 1, which is what `call{1}` needs.
- `conseq H1 H2` on a hoare goal conjoins the postconditions of two hoare triples.

$$\frac{\Phi \Rightarrow \Phi' \qquad \models c_1 \sim c_2 : \Phi' \Rightarrow \Psi' \qquad \Psi' \Rightarrow \Psi}{\models c_1 \sim c_2 : \Phi \Rightarrow \Psi}\ \texttt{conseq}$$

$$\frac{\models c_1 \sim c_2 : \Phi' \Rightarrow \Psi' \qquad \models c_1 : P_1 \Rightarrow Q_1 \qquad \models c_2 : P_2 \Rightarrow Q_2 \qquad \Phi \Rightarrow \Phi' \wedge P_1\{1\} \wedge P_2\{2\} \qquad \Psi' \wedge Q_1\{1\} \wedge Q_2\{2\} \Rightarrow \Psi}{\models c_1 \sim c_2 : \Phi \Rightarrow \Psi}\ \texttt{conseq E H1 H2}$$

$$\frac{\models c : \mathit{true} \Rightarrow \mathit{true}\ [=1] \qquad \models c : P \Rightarrow Q}{\models c : P \Rightarrow Q\ [=1]}\ \texttt{conseq ll H}$$

The second rule is sound because a Hoare postcondition holds in every final memory of its side, and so in every pair of memories the coupling can reach. The last implication must hold in all such pairs, which is why

EasyCrypt states it with the variables in $\mathrm{mod}(c_1)$ and $\mathrm{mod}(c_2)$ universally quantified.

01_conseq.ec

```
(* conseq: the shapes worth knowing.
 * Run: easycrypt compile 01_conseq.ec *)
require import AllCore.

module M = {
  var x, y : int

  proc incr() : unit = { x <- x + 1; }
  proc both() : unit = { x <- x + 1; y <- y + 1; }
  proc f(a : int) : int = { x <- x + 1; return a; }
}.

(* ---------------------------------------------------------------------- *)
(* 1. Apply a spec lemma, weakening its post / strengthening its pre.
 *    The side goals are  pre ⇒ pre'  and  post' ⇒ post. *)
lemma incr_spec (n : int) : hoare [M.incr : M.x = n ==> M.x = n + 1].
proof. by proc; auto. qed.

lemma use_spec : hoare [M.incr : 0 <= M.x ==> 0 < M.x].
proof.
  exists* M.x; elim* ⇒ n.        (* name the initial value; see 03_exlim_seq *)
  by conseq (incr_spec n) ⇒ /#.
qed.

(* ---------------------------------------------------------------------- *)
(* 2. Change the post to what the next tactic proves, keep the pre ( _ ).
 *    Typical before sim, which wants equalities. *)
lemma to_sim : equiv [M.both ~ M.both : ={M.x, M.y} ==> M.x{1} = M.x{2}].
proof.
  conseq (: _ ==> ={M.x, M.y}) ⇒ //.
  by sim.
qed.

(* Same, with the implication proved by hand instead of ⇒ // . *)
lemma to_sim' : equiv [M.both ~ M.both : ={M.x, M.y} ==> M.x{1} + M.y{1} = M.x{2} + M.y{2}].
proof.
  conseq (: _ ==> ={M.x, M.y}).
  + by move⇒ &1 &2 /> /#.        (* post implication, if not automatic *)
  by sim.
qed.

(* ---------------------------------------------------------------------- *)
(* 3. conseq E H1 H2 : glue one-sided hoare facts onto a two-sided equiv.
 *    An equiv relates {1} to {2}; it cannot say how a side's state relates to
 *    its OWN initial value.  Fix the initial values, then add a hoare triple
 *    per side.  Side goals: pre ⇒ P ∧ P1{1} ∧ P2{2}, and
 *    Q ∧ Q1{1} ∧ Q2{2} ⇒ post. *)
equiv f_eq : M.f ~ M.f : ={arg} ==> ={res}.
proof. by proc; auto. qed.

lemma f_x (n : int) : hoare [M.f : M.x = n ==> M.x = n + 1].
proof. by proc; auto. qed.

lemma glue (n1 n2 : int) :
  equiv [M.f ~ M.f : ={arg} ∧ M.x{1} = n1 ∧ M.x{2} = n2
                 ==> ={res} ∧ M.x{1} = n1 + 1 ∧ M.x{2} = n2 + 1].
proof. by conseq f_eq (f_x n1) (f_x n2). qed.

(* (_: true ==> true) = "nothing extra on this side"; it becomes a trivial goal. *)
lemma glue_right (n2 : int) :
  equiv [M.f ~ M.f : ={arg} ∧ M.x{2} = n2 ==> ={res} ∧ M.x{2} = n2 + 1].
proof. by conseq f_eq (_: true ==> true) (f_x n2). qed.
(* Here ⇒ // closed the trivial hoare goal; otherwise: + by proc; auto. *)

(* Inside a proof, when the initial value is not a lemma argument yet:
 *   exists* M.x{2}; elim* ⇒ n2.
 *   conseq f_eq (_: true ==> true) (f_x n2).                          *)

(* ---------------------------------------------------------------------- *)
(* 4. phoare = 1 from losslessness + a hoare triple (what call{i} wants). *)
lemma f_ll : islossless M.f.
proof. by proc; auto. qed.
```

```
lemma f_ph (n : int) : phoare [M.f : M.x = n ==> M.x = n + 1] = 1%r.
proof. by conseq f_ll (f_x n). qed.

(* ---------------------------------------------------------------------- *)
(* 5. Combine two hoare triples (conjoin their posts). *)
lemma f_arg (a0 : int) : hoare [M.f : arg = a0 ==> res = a0].
proof. by proc; auto. qed.

lemma f_both (n a0 : int) :
  hoare [M.f : M.x = n /\ arg = a0 ==> M.x = n + 1 /\ res = a0].
proof. by conseq (f_x n) (f_arg a0) => /#. qed.
```

# 4 call

- `call (: I)` handles a call to an abstract adversary with an invariant `I`. It leaves one goal per oracle procedure, of the form `={arg} /\ I ==> ={res} /\ I`. Directly on the procedure, the same thing is `proc I`.
- A procedure declared with `{}` in its module type may not call any oracle. `call (: true)` is then enough, since it only touches the adversary's own state and every other fact in the precondition carries over.
- For an abstract oracle procedure, or an alias like `proc get = O.get`, use `proc*; call (: true)`.
- `call (: I)` is only for abstract procedures. A concrete procedure needs a full spec, `call (: P ==> Q)`.
- `call{1} H` with `H` a phoare spec with probability 1 runs the call on the left side only. Its postcondition is all you learn about what the call did.
- `call H` with a hoare lemma `H` works as in hoare logic.

$$\frac{\forall \mathtt{o}.\ \models \mathtt{O}_1.\mathtt{o} \sim \mathtt{O}_2.\mathtt{o} : =\{arg\} \wedge I \Rightarrow =\{res\} \wedge I}{\models \mathtt{A}(\mathtt{O}_1).\mathtt{f} \sim \mathtt{A}(\mathtt{O}_2).\mathtt{f} : =\{arg, \mathtt{glob\ A}\} \wedge I \Rightarrow =\{res, \mathtt{glob\ A}\} \wedge I}\ \mathtt{call\ (:\ \ I)}$$

$$\frac{\models \mathtt{f} : P \Rightarrow Q\ [=1] \qquad \Phi \Rightarrow P[e/arg]\{1\} \wedge \forall v\, \vec{w}.\ Q[v/res, \vec{w}/\mathrm{mod}(\mathtt{f})]\{1\} \Rightarrow \Psi[v/x, \vec{w}/\mathrm{mod}(\mathtt{f})]\{1\}}{\models x \leftarrow \mathtt{f}(e) \sim \mathtt{skip} : \Phi \Rightarrow \Psi}\ \mathtt{call\{1\}\ H}$$

In the first rule `o` ranges over the oracle procedures `A` may call, $I$ must not mention `glob A`, and `A` must not be able to write the variables of $I$. A procedure declared with `{}` may call no oracle, so the rule has no premises and $I = true$ suffices, while facts about variables outside mod = `glob A` survive the call unchanged.

In the second rule the new values of everything `f` may modify are universally quantified, and $Q$ is all that is known about them. A spec that says nothing about some variable the rest of the proof needs leaves an unprovable goal, with a fresh name such as `S_R` where the old value used to be.

02_call.ec

```
(* call: the shapes worth knowing.
 * Run: easycrypt compile 02_call.ec *)
require import AllCore.

module type Orc = {
  proc get(x : int) : int
}.

(* An adversary with one state-reading procedure that may NOT call the oracle
 * ({} = empty list of allowed oracle procedures) and one that may. *)
module type Adv (O : Orc) = {
  proc save() : int {}
  proc run() : bool
}.

module O1 : Orc = {
  var n : int
  proc get(x : int) : int = { n <- n + 1; return x; }
}.

module O2 : Orc = {
  var m : int
  proc get(x : int) : int = { m <- m + 1; return x; }
```

```
}.

(* An oracle that is a mere alias of another. *)
module W (O : Orc) : Orc = {
  proc get = O.get
}.

module G (O : Orc) (A : Adv) = {
  var s : int
  proc main() : bool = {
    var b;
    s <@ A(O).save();
    b <@ A(O).run();
    return b;
  }
}.

section.

declare module A <: Adv {-O1, -O2, -G}.
declare module O <: Orc {-A}.

(* ---------------------------------------------------------------------- *)
(* 1. Abstract adversary, different oracles: call with an invariant I.
 *    One goal per oracle procedure: ={arg} ∧ I ⟹ ={res} ∧ I.
 *    The call itself gives ={res, glob A} ∧ I from ={arg, glob A} ∧ I. *)
lemma run_rel :
  equiv [G(O1, A).main ~ G(O2, A).main :
         ={glob A} ∧ O1.n{1} = O2.m{2} ⟹ ={res} ∧ O1.n{1} = O2.m{2}].
proof.
  proc.
  call (: O1.n{1} = O2.m{2}).
  + by proc; auto.                    (* the oracle goal *)
   (* save has {} : it touches nothing but glob A, so (: true) is enough and
    * every other fact in the precondition is carried over unchanged. *)
  call (: true).
  by auto.
qed.

(* The same statement proved directly on the procedure: proc I. *)
lemma run_rel' :
  equiv [A(O1).run ~ A(O2).run :
         ={glob A} ∧ O1.n{1} = O2.m{2} ⟹ ={res, glob A} ∧ O1.n{1} = O2.m{2}].
proof.
  proc (O1.n{1} = O2.m{2}) ⇒ //.
  by proc; auto.
qed.

(* ---------------------------------------------------------------------- *)
(* 2. An abstract oracle procedure, or an alias of one: proc* then call (: true).
 *    For an abstract procedure call (: true) gives ={res, glob O} from
 *    ={arg, glob O}. *)
equiv w_get : W(O).get ~ W(O).get : ={arg, glob O} ⟹ ={res, glob O}.
proof. by proc*; call (: true); auto. qed.

(* ---------------------------------------------------------------------- *)
(* 3. One-sided call: call{1} / call{2} with a phoare = 1 spec.
 *    The other side does nothing; the spec's post is all you learn. *)
module K = {
  var c : int
  proc tick() : unit = { c <- c + 1; }
  proc two() : unit = { tick(); tick(); }
  proc one() : unit = { tick(); }
}.

lemma tick_ph (n : int) : phoare [K.tick : K.c = n ⟹ K.c = n + 1] = 1%r.
proof. by proc; auto. qed.

lemma drop_one :
  equiv [K.two ~ K.one : ={K.c} ⟹ K.c{1} = K.c{2} + 1].
proof.
  proc.
  (* last calls, two-sided.  For a concrete procedure give a full spec
   * (: P ⟹ Q); the invariant form (: I) is for abstract procedures. *)
  call (: K.c{1} = K.c{2} + 1 ⟹ K.c{1} = K.c{2} + 1).
```

```
  + by proc; auto.
  (* the remaining left-only call: fix K.c{1}, then call{1} *)
  exlim K.c{1} ⇒ n.
  call{1} (tick_ph n).
  by auto ⇒ /#.
qed.

(* A lossless spec drops a call whose effect you don't care about:
 *   call{1} some_ll.      where some_ll : islossless P.f
 * You lose every fact about what P.f writes. *)

(* ---------------------------------------------------------------------- *)
(* 4. call with a spec lemma instead of an invariant (hoare or equiv). *)
lemma tick_h (n : int) : hoare [K.tick : K.c = n ⟹ K.c = n + 1].
proof. by proc; auto. qed.

lemma two_h (n : int) : hoare [K.two : K.c = n ⟹ K.c = n + 2].
proof.
  proc.
  call (tick_h (n + 1)).
  call (tick_h n).
  by auto ⇒ /#.
qed.

end section.
```

# 5 exlim and seq

- `exlim e ⇒ v` binds `v` to the value of `e` in the precondition of the current goal. That precondition describes the state at the start of the program that is left, not at the point where you want to use `v`.
- The typical mistake is an `exlim` at the top of a proof whose program changes the value before the call that needs it. The call's precondition then asks for an equality that no longer holds.
- The fix is to cut the program first, so that the call becomes the start of what is left. `seq` cuts at any point, and `sp` consumes leading assignments.
- In a relational goal, say which side, as in `exlim x{1}, M.y{2} ⇒ a b`.
- An `exlim` at the top is fine when nothing before the call changes the value, because then the value at the start and the value at the call coincide.

$$\frac{\models c_1 \sim c_2 : \Phi \Rightarrow \Theta \qquad \models c_1' \sim c_2' : \Theta \Rightarrow \Psi}{\models c_1; c_1' \sim c_2; c_2' : \Phi \Rightarrow \Psi}\ \texttt{seq} \qquad \frac{\forall v.\ \models c_1 \sim c_2 : \Phi \wedge e = v \Rightarrow \Psi}{\models c_1 \sim c_2 : \Phi \Rightarrow \Psi}\ \texttt{exlim e => v}$$

In the second rule $v$ is fresh, and $e$ is evaluated in the memories $\Phi$ talks about, which are the initial memories of all of $c_1$ and $c_2$. To name a value in the middle of a program, apply `seq` first and then `exlim` in its second premise, where the middle of the program has become the start of $c_1'$ and $c_2'$.

03_exlim_seq.ec

```
(* exlim / exists* and seq.
 * Run: easycrypt compile 03_exlim_seq.ec
 *
 * THE RULE: exlim e ⇒ v  binds v to the value of e in the PRECONDITION of
 * the current goal, i.e. at the START of the program that is left.  If the
 * program changes e before the point where you need v, first cut the program
 * so that point becomes the start (seq, sp), then exlim.
 *
 *   exlim e1, e2 ⇒ a b.          same as   exists* e1, e2; elim* ⇒ a b.
 *   exlim x{1}, M.y{2} ⇒ a b.    relational goals: say which side.
 *)
require import AllCore.

module P = {
  var x : int
  proc inc() : unit = { x <- x + 1; }
  proc main() : unit = { x <- 5; inc(); }
  proc main2() : unit = { inc(); }
}.
```

```
20
21 lemma inc_h (n : int) : hoare [P.inc : P.x = n ⟹ P.x = n + 1].
22 proof. by proc; auto. qed.
23
24 lemma inc_ph (n : int) : phoare [P.inc : P.x = n ⟹ P.x = n + 1] = 1%r.
25 proof. by proc; auto. qed.
26
27 (* ---------------------------------------------------------------------- *)
28 (* WRONG: exlim at the top binds the value BEFORE x <- 5, so the call's
29  * precondition P.x = n cannot be met:
30  *
31  *   proc. exlim P.x ⇒ n. call (inc_h n).   (* stuck: need 5 = n *)
32  *
33  * RIGHT: cut first, so the call is at the start of what is left.          *)
34 lemma main_h : hoare [P.main : true ⟹ P.x = 6].
35 proof.
36   proc.
37   seq 1 : (P.x = 5); first by auto.   (* hoare seq: seq <#stmts> : (P) *)
38   exlim P.x ⇒ n.                      (* now n is the value at the call *)
39   call (inc_h n).
40   by auto ⇒ /#.
41 qed.
42
43 (* For straight-line assignments at the start, sp does the cut for you. *)
44 lemma main_h' : hoare [P.main : true ⟹ P.x = 6].
45 proof.
46   proc; sp.                           (* pre is now P.x = 5 *)
47   exlim P.x ⇒ n.
48   call (inc_h n).
49   by auto ⇒ /#.
50 qed.
51
52 (* ---------------------------------------------------------------------- *)
53 (* Relational version: a one-sided call needs the ghost at the call point.
54  * equiv seq: seq <#left> <#right> : (P).  #pre = the current precondition. *)
55 lemma main_eq : equiv [P.main ~ P.main2 : true ⟹ P.x{1} = 6].
56 proof.
57   proc.
58   seq 1 0 : (P.x{1} = 5); first by auto.
59   exlim P.x{1} ⇒ n.
60   call{1} (inc_ph n).
61   (* right side still has its call; drop it one-sided too (no ghost needed) *)
62   call{2} (: true ⟹ true); first by proc; auto.
63   by auto ⇒ /#.
64 qed.
65
66 (* ---------------------------------------------------------------------- *)
67 (* Calls are processed from the END of the program.  exlim at the top is fine
68  * when nothing before the call changes the value: then "value at the start"
69  * and "value at the call" coincide. *)
70 lemma twice (n : int) :
71   equiv [P.main2 ~ P.main2 : P.x{1} = n ∧ P.x{2} = n ⟹ ={P.x}].
72 proof.
73   proc.
74   exlim P.x{1} ⇒ v.                   (* fine: inc is the first statement *)
75   call{1} (inc_ph v).
76   exlim P.x{2} ⇒ w.
77   call{2} (inc_ph w).
78   by auto ⇒ /#.
79 qed.
80
81 (* ---------------------------------------------------------------------- *)
82 (* Other cutting tools
83  *   seq 3 3 : (#pre ∧ ={x}).        keep the old facts and add one
84  *   seq 1 1 : (Q); first by ... .    discharge the prefix inline
85  *   sp / sp 1 1                      consume leading assignments into the pre
86  *   wp                               consume trailing assignments into the post
87  *   swap{1} 3 -2                     move left statement 3 up by 2 (needs
88  *                                    syntactic independence)                *)
```

# 6 Probabilities

- `rewrite Pr[lemma]` applies a distribution lemma to a probability term. The useful ones are `mu_le1`, `mu_sub`, `mu_not`, `mu_or` and `mu_split E`.
- $\Pr[E] = 0$ follows from a hoare triple on $\neg E$, by `byphoare`, then `hoare`.
- A phoare spec with probability 1 follows from a fact of the form $\Pr[\ldots] = 1$ that holds at every memory, by `bypr`.
- A hoare spec follows from the same fact. `bypr` turns the goal into $\Pr[\neg Q] = 0$, and $\Pr[\neg Q] = \Pr[\mathit{true}] - \Pr[Q]$ with $1 = \Pr[Q] \le \Pr[\mathit{true}] \le 1$. No losslessness is needed.
- Inside an event written in a proof, `X{m}` for the memory bound by `bypr` is read as the current `X`. Rather than retyping the event, rewrite with the hypothesis that already contains it, as in `rewrite -h1 Pr[mu_sub]`.
- `byequiv lemma` needs the lemma's pre- and postcondition to match exactly. State the spec in `byequiv`, discharge it with `conseq`, and use `symmetry` when the games sit on the other sides.
- `smt` treats probability terms as opaque reals, so arithmetic over several of them works as long as the terms are syntactically identical.

$$\frac{\models c_1 \sim c_2 : \Phi \Rightarrow \Psi \qquad \Phi(m_1, m_2) \qquad \Psi \Rightarrow (E_1\{1\} \Rightarrow E_2\{2\})}{\Pr[c_1, m_1 : E_1] \le \Pr[c_2, m_2 : E_2]}\ \texttt{byequiv}$$

$$\frac{\models c_2 \sim c_1 : \Phi^{-1} \Rightarrow \Psi^{-1}}{\models c_1 \sim c_2 : \Phi \Rightarrow \Psi}\ \texttt{symmetry} \qquad \frac{\forall m.\ P(m) \Rightarrow \Pr[c, m : Q] = p}{\models c : P \Rightarrow Q\ [=p]}\ \texttt{bypr (phoare)}$$

$$\frac{\forall m.\ P(m) \Rightarrow \Pr[c, m : \neg Q] = 0}{\models c : P \Rightarrow Q}\ \texttt{bypr (hoare)} \qquad \frac{\models c : P \Rightarrow \neg E \qquad P(m)}{\Pr[c, m : E] = 0}\ \texttt{byphoare; hoare}$$

With $\Leftrightarrow$ in place of $\Rightarrow$ in the last premise, `byequiv` gives equality. $\Phi^{-1}$ is $\Phi$ with $\{1\}$ and $\{2\}$ exchanged.

04_pr.ec

```
(* Pr[...] rewriting, and moving between Pr, hoare and phoare.
 * Run: easycrypt compile 04_pr.ec *)
require import AllCore Distr DBool.

module T = {
  var x : int
  proc flip() : bool = { var b; b <$ {0,1}; return b; }
  proc keep() : unit = { }
}.

(* ------------------------------------------------------------------------ *)
(* 1. The rewrite rules: Pr[lemma] applies a distribution lemma to a Pr term. *)
lemma le1 &m : Pr[T.flip() @ &m : res] <= 1%r.
proof. by rewrite Pr[mu_le1]. qed.

lemma sub &m : Pr[T.flip() @ &m : res /\ T.x = 0] <= Pr[T.flip() @ &m : res].
proof. by rewrite Pr[mu_sub]. qed.            (* event inclusion *)

lemma not &m :
  Pr[T.flip() @ &m : !res] = Pr[T.flip() @ &m : true] - Pr[T.flip() @ &m : res].
proof. by rewrite Pr[mu_not]. qed.

lemma or &m :
  Pr[T.flip() @ &m : res \/ T.x = 0] =
  Pr[T.flip() @ &m : res] + Pr[T.flip() @ &m : T.x = 0]
  - Pr[T.flip() @ &m : res /\ T.x = 0].
proof. by rewrite Pr[mu_or]. qed.

lemma split_ex &m :
  Pr[T.flip() @ &m : res] =
  Pr[T.flip() @ &m : res /\ T.x = 0] + Pr[T.flip() @ &m : res /\ !(T.x = 0)].
proof. by rewrite Pr[mu_split (T.x = 0)]. qed.

(* NOTE: `split` is a keyword: `lemma split` and `have split : ...` are both
 * parse errors.  Same for other tactic names.  Use hsplit, split_ex, ... *)

(* ------------------------------------------------------------------------ *)
(* 2. Pr[E] = 0 from a hoare triple on "not E". *)
```

```
lemma never &m : Pr[T.keep() @ &m : T.x ≠ T.x{m}] = 0%r.
proof. byphoare (_: T.x = T.x{m} ⟹ _) ⇒ //; hoare; proc; auto. qed.

(* ---------------------------------------------------------------------- *)
(* 3. phoare = 1 from a Pr = 1 fact that holds at every memory: bypr. *)
lemma keep_pr &m : Pr[T.keep() @ &m : T.x = T.x{m}] = 1%r.
proof. byphoare (_: T.x = T.x{m} ⟹ T.x = T.x{m}) ⇒ //; proc; auto. qed.

lemma keep_ph (n : int) : phoare [T.keep : T.x = n ⟹ T.x = n] = 1%r.
proof. by bypr ⇒ &m <-; exact (keep_pr &m). qed.

(* ---------------------------------------------------------------------- *)
(* 4. hoare from a Pr = 1 fact: bypr turns the goal into Pr[not Q] = 0.
 *    Pr[not Q] = Pr[true] - Pr[Q] = Pr[true] - 1, and 1 = Pr[Q] ≤ Pr[true] ≤ 1.
 *    No losslessness needed.
 *
 *    PITFALL: inside a Pr[...] event written in a proof, T.x{m} for the
 *    memory bound by bypr is read as the CURRENT value (the event becomes
 *    T.x = T.x).  Don't write the event by hand; get it from the hypothesis,
 *    as in  rewrite -h1 Pr[mu_sub]. *)
lemma keep_h (n : int) : hoare [T.keep : T.x = n ⟹ T.x = n].
proof.
  bypr ⇒ &m <-.
  have h1 := keep_pr &m.
  have h2 : Pr[T.keep() @ &m : true] ≤ 1%r by rewrite Pr[mu_le1].
  have h3 : 1%r ≤ Pr[T.keep() @ &m : true] by rewrite -h1 Pr[mu_sub].
  by rewrite Pr[mu_not] h1; smt().
qed.

(* ---------------------------------------------------------------------- *)
(* 5. byequiv: a lemma must match the goal's pre/post EXACTLY.  State the
 *    spec you want in byequiv, then conseq the lemma into it.  If the games
 *    are on the other sides, symmetry flips the goal ({1} and {2} swap). *)
module U = {
  proc f() : bool = { var b; b <$ {0,1}; return b; }
  proc g() : bool = { var b; b <$ {0,1}; return !b; }
}.

equiv fg : U.f ~ U.g : true ⟹ res{1} = !res{2}.
proof. by proc; auto. qed.                        (* couples b{1} = b{2} *)

lemma fg_pr &m : Pr[U.f() @ &m : res] ≤ Pr[U.g() @ &m : !res].
proof.
  byequiv (: true ⟹ res{1} = !res{2}) ⇒ //.    (* ⇒ // closed pre/post *)
  by conseq fg.
qed.

lemma gf_pr &m : Pr[U.g() @ &m : !res] ≤ Pr[U.f() @ &m : res].
proof.
  byequiv (: true ⟹ res{2} = !res{1}) ⇒ //.
  by symmetry; conseq fg ⇒ /#.
qed.

(* ---------------------------------------------------------------------- *)
(* 6. Pr arithmetic with smt: hypotheses about Pr terms are fine, smt treats
 *    the Pr terms as opaque reals.  Keep the terms syntactically identical. *)
lemma eq_from_two &m : Pr[U.f() @ &m : res] = Pr[U.g() @ &m : !res].
proof. by have := fg_pr &m; have := gf_pr &m; smt(). qed.
```

# 7 Up to bad

The call rule takes three components, `call (: B, I, J)`.

| | |
|---|---|
| `B` | the bad event, read on the right memory and written without `{2}` |
| `I` | the relational invariant while ¬`B`, which must include the equality of the flag |
| `J` | what still holds once `B` is set, preserved by each side alone (optional) |

It generates a goal saying that the adversary terminates when its oracles do, and then three goals for every oracle procedure `o`.

1. `equiv [o ~ o : !B{2} ∧ ={arg} ∧ I ⟹ if B{2} then J else ={res} ∧ I]`.
2. `forall &2, B{2} ⇒ phoare [o{1} : J ⟹ J] = 1%r`, for the left side after bad.
3. `forall &1, phoare [o{2} : B ∧ J ⟹ B ∧ J] = 1%r`, which says the right side keeps bad.

The code before the call is left with the postcondition `if B{2} then J else (={res, glob A} ∧ I)`.

- Choose `J` so that it implies bad stays set. If some oracle could unset bad, for instance by overwriting the record that bad is computed from, put into `J` whatever switches that oracle off.
- If you only need `!B{2} ⇒ ={res}` and not the equality of the flags afterwards, drop `J`. The goals after bad then only ask for termination.
- From the equivalence, one `byequiv` gives $\Pr_1[E] \le \Pr_2[E \vee bad] \le \Pr_2[E] + \Pr_2[bad]$.
- If $\Pr_2[bad] = 0$, which a hoare triple can show, a second `byequiv` in the other direction gives $\Pr_2[E \wedge \neg bad] \le \Pr_1[E]$. Splitting $\Pr_2[E]$ on bad then yields equality.

Writing $\mathcal{I}(X)$ for **if** $B\{2\}$ **then** $J$ **else** $(=\{X\} \wedge I)$, the rule reads as follows.

$$\frac{\begin{array}{c}\texttt{A} \text{ is lossless when its oracles are} \\ \forall \texttt{o}.\ \models \texttt{O}_1.\texttt{o} \sim \texttt{O}_2.\texttt{o} : \neg B\{2\} \wedge =\{arg\} \wedge I \Rightarrow \mathcal{I}(res) \\ \forall \texttt{o}.\ \forall m_2.\ B(m_2) \Rightarrow \big(\models \texttt{O}_1.\texttt{o} : J \Rightarrow J\ [=1]\big) \qquad \forall \texttt{o}.\ \forall m_1.\ \models \texttt{O}_2.\texttt{o} : B \wedge J \Rightarrow B \wedge J\ [=1]\end{array}}{\models \texttt{A}(\texttt{O}_1).\texttt{f} \sim \texttt{A}(\texttt{O}_2).\texttt{f} : \mathcal{I}(arg, \texttt{glob A}) \Rightarrow \mathcal{I}(res, \texttt{glob A})}\ \texttt{call (: B, I, J)}$$

$$\frac{\models c_1 \sim c_2 : \Phi \Rightarrow =\{B\} \wedge (\neg B\{2\} \Rightarrow =\{res\}) \qquad \Phi(m_1, m_2)}{\Pr[c_1, m_1 : E] \le \Pr[c_2, m_2 : E] + \Pr[c_2, m_2 : B]}\ \text{up to bad}$$

In the third and fourth premises $J$ is a relation, and the memory of the side that does not run is the fixed $m_2$ or $m_1$. The second rule is `byequiv` followed by $\Pr[E \vee B] \le \Pr[E] + \Pr[B]$, where $E$ is an event on the result.

05_upto_bad.ec

```
(* Up-to-bad: two games that agree until a flag is raised.
 * Run: easycrypt compile 05_upto_bad.ec
 *
 * The call rule has three parts:
 *
 *   call (: B, I, J).
 *     B : the bad event, read on the RIGHT memory (written without {2})
 *     I : relational invariant while not B
 *     J : what still holds once B happened (relational; optional)
 *
 * It generates, for the adversary and then for EACH oracle procedure o:
 *   - the adversary terminates when its oracles do
 *   - equiv [o ~ o : !B{2} ∧ ={arg} ∧ I ⟹ if B{2} then J else ={res} ∧ I]
 *   - forall &2, B{2} ⇒ phoare [o{1} : J ⟹ J] = 1      (left, after bad)
 *   - forall &1,        phoare [o{2} : B ∧ J ⟹ B ∧ J] = 1  (right keeps bad)
 * and leaves the code before the call with post
 *   if B{2} then J else (={res, glob A} ∧ I).
 *
 * Choosing I: include ={flag} itself, or nothing says the LEFT flag is
 * still down while the right one is (you then can't conclude ={Flag.bad}).
 *
 * Choosing J: it must be preserved by each side ALONE, and it must imply
 * that B stays set.  If an oracle could un-set B (e.g. overwrite the record
 * B is computed from) put in J whatever switches that oracle off.
 *)
require import AllCore Distr.

module type Orc = {
  proc f(x : int) : int
}.

module type Adv (O : Orc) = {
  proc run() : bool
}.

module Flag = {
  var bad : bool
  var s   : int
}.
```

```
40
41  (* Two oracles that differ only on the point s; both raise the flag there.
42   * Nonnegative queries only, so with s < 0 the flag can never be raised. *)
43  module OL : Orc = {
44    proc f(x : int) : int = {
45      if (0 ≤ x ∧ x = Flag.s) { Flag.bad <- true; }
46      return x;
47    }
48  }.
49
50  module OR : Orc = {
51    proc f(x : int) : int = {
52      var r;
53      r <- x;
54      if (0 ≤ x ∧ x = Flag.s) { Flag.bad <- true; r <- 0; }
55      return r;
56    }
57  }.
58
59  module Game (O : Orc) (A : Adv) = {
60    proc main(s0 : int) : bool = {
61      var b;
62      Flag.bad <- false;
63      Flag.s <- s0;
64      b <@ A(O).run();
65      return b;
66    }
67  }.
68
69  section.
70
71  declare module A <: Adv {-Flag}.
72
73  declare axiom A_ll (O <: Orc {-A}) : islossless O.f ⇒ islossless A(O).run.
74
75  (* ---------------------------------------------------------------------- *)
76  (* 1. The up-to-bad equivalence. *)
77  equiv upto : Game(OL, A).main ~ Game(OR, A).main :
78    ={glob A, arg} ⟹ ={Flag.bad} ∧ (!Flag.bad{2} ⇒ ={res}).
79  proof.
80    proc.
81    call (: Flag.bad, ={Flag.s, Flag.bad}, Flag.bad{1}).
82    + exact A_ll.                                  (* adversary terminates *)
83    + by proc; auto ⇒ /#.                          (* oracle, before bad *)
84    + by move⇒ &2 _; proc; auto.                   (* left oracle, after bad *)
85    + by move⇒ &1; proc; auto.                     (* right oracle keeps bad *)
86    by auto ⇒ /#.
87  qed.
88
89  (* If you only need !bad{2} ⇒ ={res} (not ={Flag.bad}), drop J:
90   *   call (: Flag.bad, ={Flag.s, Flag.bad}).
91   * and the "after bad" goals only ask for termination. *)
92
93  (* ---------------------------------------------------------------------- *)
94  (* 2. The usual bound: Pr1[E] ≤ Pr2[E] + Pr2[bad]. *)
95  lemma upto_bound &m (s0 : int) :
96    Pr[Game(OL, A).main(s0) @ &m : res] ≤
97    Pr[Game(OR, A).main(s0) @ &m : res] + Pr[Game(OR, A).main(s0) @ &m : Flag.bad].
98  proof.
99    have le : Pr[Game(OL, A).main(s0) @ &m : res] ≤
100             Pr[Game(OR, A).main(s0) @ &m : res ∨ Flag.bad].
101   + byequiv (: ={glob A, arg} ⟹ ={Flag.bad} ∧ (!Flag.bad{2} ⇒ ={res})) ⇒ //.
102     + by conseq upto.
103     by move⇒ &1 &2 /#.
104   have hor : Pr[Game(OR, A).main(s0) @ &m : res ∨ Flag.bad] ≤
105              Pr[Game(OR, A).main(s0) @ &m : res] +
106              Pr[Game(OR, A).main(s0) @ &m : Flag.bad].
107   + by rewrite Pr[mu_or]; smt(mu_bounded).      (* drop the - Pr[res ∧ bad] *)
108   smt().
109 qed.
110
111 (* ---------------------------------------------------------------------- *)
112 (* 3. When bad provably never happens: Pr[bad] = 0 from a hoare triple on the
113  *    right game, then equality from two one-directional byequivs. *)
114 lemma no_bad &m (s0 : int) :
```

```
  s0 < 0 ⇒ Pr[Game(OR, A).main(s0) @ &m : Flag.bad] = 0%r.
proof.
  move⇒ hs; byphoare (_: arg = s0 ⟹ _) ⇒ //; hoare.
  proc.
  call (: !Flag.bad ∧ Flag.s < 0); first by proc; auto ⇒ /#.
  by auto ⇒ /#.
qed.

lemma upto_eq &m (s0 : int) :
  s0 < 0 ⇒
  Pr[Game(OL, A).main(s0) @ &m : res] = Pr[Game(OR, A).main(s0) @ &m : res].
proof.
  move⇒ hs; have nb := no_bad &m s0 hs.
  (* Pr1[res] ≤ Pr2[res ∨ bad] *)
  have le1 : Pr[Game(OL, A).main(s0) @ &m : res] ≤
             Pr[Game(OR, A).main(s0) @ &m : res ∨ Flag.bad].
  + byequiv (: ={glob A, arg} ⟹ ={Flag.bad} ∧ (!Flag.bad{2} ⇒ ={res})) ⇒ //.
    + by conseq upto.
    by move⇒ &1 &2 /#.
  (* Pr2[res ∧ !bad] ≤ Pr1[res]: the games are now on the other sides *)
  have le2 : Pr[Game(OR, A).main(s0) @ &m : res ∧ !Flag.bad] ≤
             Pr[Game(OL, A).main(s0) @ &m : res].
  + byequiv (: ={glob A, arg} ⟹ ={Flag.bad} ∧ (!Flag.bad{1} ⇒ ={res})) ⇒ //.
    + by symmetry; conseq upto ⇒ /#.
    by move⇒ &1 &2 /#.
  (* arithmetic *)
  have hor : Pr[Game(OR, A).main(s0) @ &m : res ∨ Flag.bad] ≤
             Pr[Game(OR, A).main(s0) @ &m : res] +
             Pr[Game(OR, A).main(s0) @ &m : Flag.bad].
  + by rewrite Pr[mu_or]; smt(mu_bounded).
  have hsplit : Pr[Game(OR, A).main(s0) @ &m : res] =
                Pr[Game(OR, A).main(s0) @ &m : res ∧ Flag.bad] +
                Pr[Game(OR, A).main(s0) @ &m : res ∧ !Flag.bad].
  + by rewrite Pr[mu_split Flag.bad].
  have hsub : Pr[Game(OR, A).main(s0) @ &m : res ∧ Flag.bad] ≤
              Pr[Game(OR, A).main(s0) @ &m : Flag.bad].
  + by rewrite Pr[mu_sub].
  smt(mu_bounded).
qed.

end section.
```

# 8 Dropping one-sided calls

A rewinding reduction runs the adversary once more than the game it is compared with, and that extra run is dropped on one side only.

1. For every call you drop, get a phoare spec with probability 1 that says what it does to the state you still care about. It comes from losslessness and a hoare fact through `conseq ll H`, or from an axiom of the form $\Pr[\ldots] = 1$ through `bypr`. If the axiom is existential, as a rewinding axiom usually is, these specs must be `have`s inside the proof, after `elim`.
2. Fix, with `exlim`, the values those specs need, at a point where they are right.
3. Run `call{1}` with each spec, from the last dropped call backwards.
4. Close with `auto` and `smt`.

The state-reading procedures are declared with `{}`, so they cannot call the oracle. They stand for the reduction taking a snapshot of the adversary, not for something the adversary does.

Each dropped call is one application of `call{1} H` from Section 4. For a discarded run between a save and a restore, the derivation is the following chain, read from the bottom.

$$\dfrac{\dfrac{\dfrac{\Phi \wedge \texttt{glob A}\{1\} = g \Rightarrow \Psi}{\models \texttt{load}(s) \sim \texttt{skip} : s\{1\} = f(g) \wedge \Phi \Rightarrow \Psi}\ \texttt{call\{1\} (phl g)}}{\models \texttt{run}(); \texttt{load}(s) \sim \texttt{skip} : s\{1\} = f(g) \wedge \Phi \Rightarrow \Psi}\ \texttt{call\{1\} (phr k)}}{\models s \leftarrow \texttt{save}(); \texttt{run}(); \texttt{load}(s) \sim \texttt{skip} : \texttt{glob A}\{1\} = g \wedge \Phi \Rightarrow \Psi}\ \texttt{call\{1\} (phs g)}$$

Here $\Phi$ mentions only state that `run` leaves alone, which is what the spec `phr` provides, and $\texttt{load}(f(g))$

restores `glob A` to $g$ whatever `run` did to it.

06_one_sided.ec

```
(* Dropping one-sided calls to an abstract adversary (e.g. a discarded run in
 * a rewinding reduction).
 * Run: easycrypt compile 06_one_sided.ec
 *
 * Recipe:
 *   1. for every call you drop, a phoare [...] = 1%r spec saying what it does
 *      to the state you still care about;
 *   2. exlim the values those specs need, at a point where they are right
 *      (see 03_exlim_seq.ec);
 *   3. call{1} each spec, from the LAST dropped call backwards;
 *   4. close with auto / smt.
 *
 * Where the phoare specs come from:
 *   - losslessness + a hoare fact:  conseq ll H          (01_conseq.ec, 4.)
 *   - an axiom stated with Pr[...] = 1%r:  bypr => &m ...; exact ...
 *)
require import AllCore.

module type Orc = {
  proc get(x : int) : int
}.

(* save and load may not call the oracle ({}): they are the reduction's
 * snapshot of the adversary, not adversary actions. *)
module type Adv (O : Orc) = {
  proc save() : int {}
  proc load(s : int) : unit {}
  proc run() : bool
}.

(* An oracle that only reads its state, so a run leaves it unchanged. *)
module Cnt : Orc = {
  var k : int
  proc get(x : int) : int = { return x + k; }
}.

module G1 (A : Adv) = {
  var st : int
  proc main() : bool = {
    var b1, b2;
    st <@ A(Cnt).save();
    b1 <@ A(Cnt).run();           (* discarded run *)
    A(Cnt).load(st);
    b2 <@ A(Cnt).run();
    return b2;
  }
}.

module G2 (A : Adv) = {
  proc main() : bool = {
    var b;
    b <@ A(Cnt).run();
    return b;
  }
}.

section.

declare module A <: Adv {-Cnt, -G1}.

declare axiom A_ll (O <: Orc {-A}) : islossless O.get => islossless A(O).run.

(* Rewindability, stated with probabilities. *)
declare axiom A_rew :
  exists (f : glob A -> int),
    (forall (O <: Orc) &m,
       Pr[A(O).save() @ &m : (glob A) = (glob A){m} /\ res = f (glob A){m}] = 1%r) /\
    (forall (O <: Orc) &m (x : glob A) (s : int),
       s = f x => Pr[A(O).load(s) @ &m : (glob A) = x] = 1%r).

lemma drop_run : equiv [G1(A).main ~ G2(A).main : ={glob A, Cnt.k} ==> ={res}].
proof.
```

```
  (* 1. the specs.  f only exists after elim, so these are haves. *)
  elim A_rew ⇒ f [hs hl].
  have phs : forall g, phoare [A(Cnt).save : (glob A) = g ==> (glob A) = g ∧ res = f g] = 1%r.
  + by move⇒ g; bypr ⇒ &m <-; exact (hs Cnt &m).
  have phl : forall g, phoare [A(Cnt).load : arg = f g ==> (glob A) = g] = 1%r.
  + by move⇒ g; bypr ⇒ &m hs'; exact (hl Cnt &m g arg{m} hs').
  have phr : forall k0, phoare [A(Cnt).run : Cnt.k = k0 ==> Cnt.k = k0] = 1%r.
  + move⇒ k0; conseq (A_ll Cnt _) (_: Cnt.k = k0 ==> Cnt.k = k0) ⇒ //.
    (* goals come out in this order: the hoare fact, then the side condition
     * of A_ll.  When unsure, look: easycrypt llm, then GOALS ALL. *)
    + by proc (Cnt.k = k0) ⇒ //; proc; auto.      (* run leaves Cnt.k *)
    by proc; auto.                                (* Cnt.get is lossless *)
  proc.
  (* the kept run, two-sided *)
  call (: ={Cnt.k}); first by proc; auto.
  (* 2. nothing before the dropped calls changes glob A or Cnt.k, so exlim
   *    at the top gives the right values *)
  exlim (glob A){1}, Cnt.k{1} ⇒ g k0.
  (* 3. backwards: load, then the run, then save *)
  call{1} (phl g).
  call{1} (phr k0).
  call{1} (phs g).
  (* 4. *)
  by auto ⇒ /#.
qed.

end section.
```

# 9 Expected values: `ehoare`

- Pre and post are of type `xreal`. `x%xr` injects an `int`, `real` or `bool`, and `b `|` x` is $x$ if $b$ holds and $\infty$ otherwise. In a precondition it is an assumption, since an $\infty$ bound is always met.
- After `wp`, `rnd` and `skip` the goal is an inequality between `xreal`s. `xle_cxr_r` assumes the condition on the right, `xle_cxr_l` discharges the one on the left, `xle_cxr` does both, and `xle_rle` goes back to reals.
- Take the conditions out with `xle_cxr_r` before simplifying an expectation; in the other order the goal grew until `apply` hung.
- `call /F H` uses $\models_{\mathbb{E}} f : F \circ P \Rightarrow F \circ Q$ from $H : \models_{\mathbb{E}} f : P \Rightarrow Q$, for $F$ concave and increasing. With $F = \lambda v.\ A \texttt{ `|` } v$ it frames a fact $A$ about variables the call does not write; with $F = \lambda v.\ p \cdot v$ (written `p%xr * v`) it scales the bound.
- An oracle can be charged through an abstract adversary: the invariant of `call (: I)` is a potential that keeps the unspent budget, and each oracle call pays for its expected cost out of it.
- `byehoare` turns the judgement into $\Pr[\ldots] \le p$ when the post dominates the indicator of the event.

$$\frac{\models_{\mathbb{E}} c : f \Rightarrow \lambda m.\ \mathbb{E}_{v \sim d}\,[g(m[x \mapsto v])]}{\models_{\mathbb{E}} c;\ x \xleftarrow{\$} d : f \Rightarrow g}\ \texttt{rnd} \qquad \frac{\models_{\mathbb{E}} \texttt{f} : P \Rightarrow Q \qquad F \text{ concave, increasing}}{\models_{\mathbb{E}} \texttt{f} : F \circ P \Rightarrow F \circ Q}\ \texttt{call /F H}$$

$$\frac{\models_{\mathbb{E}} c : b \texttt{ `|` } I \Rightarrow I \qquad \forall m.\ g(m) \le (\neg b \texttt{ `|` } I)(m)}{\models_{\mathbb{E}} \texttt{while } b \texttt{ do } c : I \Rightarrow g}\ \texttt{while I}$$

$$\frac{\models_{\mathbb{E}} c : f \Rightarrow g \qquad \mathbf{1}_E \le g \qquad f(m) = p}{\Pr[c@m : E] \le p}\ \texttt{byehoare}$$

07_ehoare.ec

```
(* ehoare: bounding EXPECTED values (expectation Hoare logic).
 * Run: easycrypt compile 07_ehoare.ec
 *
 *   ehoare [ M.f : pre ==> post ]
 *
 * pre and post are xreal-valued (xreal = non-negative reals + oo).  The
 * judgment says: for every initial memory, the expected value of post over
 * the final memories is at most pre (evaluated in the initial memory).
 * Read it as "post is a potential; pre pays for it".
```

```
 *
 * Vocabulary (theory Xreal):
 *   x%xr          real / int / bool into xreal (negative reals become 0,
 *                 bools become 0 or 1)
 *   b `|` x       "x if b, oo otherwise": puts a boolean fact in a pre.
 *                 An oo pre is trivially true, so b is an assumption.
 *   Ep d f        expectation of f : 'a → xreal under d
 *   xle_cxr_r     (b ⇒ x ≤ y) ⇒ x ≤ (b `|` y)      (use the fact b)
 *   xle_cxr_l     b ⇒ x ≤ y  ⇒ (b `|` x) ≤ y       (discharge b)
 *   xle_cxr       both at once
 *   xle_rle       0 ≤ x ≤ y ⇒ x%xr ≤ y%xr          (back to reals)
 *   xlexx         x ≤ x
 *   Ep_mu         Ep d (fun x ⇒ (p x)%xr) = (mu d p)%xr
 *   EpC           Ep d (fun _ ⇒ c) = (weight d) ** c
 *   Ep_dbool      Ep {0,1} f = 0.5 ** f true + 0.5 ** f false
 *
 * After proc; wp; skip (or auto) the goal is   forall &hr, post' ≤ pre
 * (an xreal inequality), which smt / the lemmas above close. *)
require import AllCore Distr DBool Xreal.
require import StdOrder.
import RealOrder.

module M = {
  var hits : int

  proc coin() : bool = {
    var b;
    b <$ {0,1};
    return b;
  }

  proc incr(x : int) : int = { return x + 1; }

  proc flip() : unit = {
    var b;
    b <$ {0,1};
    if (b) hits <- hits + 1;
  }

  proc run(n : int) : unit = {
    var i;
    i <- 0;
    while (i < n) {
      flip();
      i <- i + 1;
    }
  }
}.

(* ------------------------------------------------------------------------ *)
(* 1. rnd turns the post into an expectation over the distribution.
 *    Close with a law-specific lemma (Ep_dbool: the two-point average) or
 *    with Ep_mu for an indicator.  Note: rewrite Ep_mu did not match
 *    fun b ⇒ b%xr here (higher-order pattern); Ep_dbool does. *)
lemma coin_e : ehoare [M.coin : (1%r / 2%r)%xr ==> res%xr].
proof.
  proc; rnd; skip ⇒ &hr /=.
  (* goal:  Ep {0,1} (fun b ⇒ b%xr) ≤ (1/2)%xr *)
  by rewrite Ep_dbool.
qed.

(* ------------------------------------------------------------------------ *)
(* 2. ehoare → Pr bound: byehoare (: pre ==> post).
 *    The event is res; the post must be >= the indicator (res%xr here). *)
module G = {
  proc main() : bool = { var b; b <@ M.coin(); return b; }
}.

lemma pr_coin &m : Pr[G.main() @ &m : res] ≤ 1%r / 2%r.
proof.
  (* side goal: the pre is a non-negative real (0 ≤ 1/2) *)
  byehoare (: (1%r / 2%r)%xr ==> res%xr) ⇒ //; last by smt().
  by proc; call coin_e; skip ⇒ &hr; exact xlexx.
qed.
```

```
(* ---------------------------------------------------------------------- *)
(* 3. "Exact" specs for any potential K.  For a deterministic procedure,
 *    ehoare [f : K (spec arg) ==> K res]  for every K  is as strong as the
 *    functional spec; it is the form that composes in ehoare proofs.
 *    (Simon.ec: evaluate_expectation, find_partner_expectation, ...) *)
lemma incr_K (K : int -> xreal) : ehoare [M.incr : K (x + 1) ==> K res].
proof. by proc; skip => &hr; apply xlexx. qed.

(* ---------------------------------------------------------------------- *)
(* 4. call with a TRANSFORMER:  call /(fun v => F v) lemma.
 *    From  ehoare [f : P ==> Q]  it uses  ehoare [f : F P ==> F Q]  when F
 *    is concave and increasing (EasyCrypt asks for that as a side goal,
 *    usually closed automatically).  Two transformers do most of the work:
 *
 *      (fun v => p%xr * v)      scale a bound by p       (LateSample.ec)
 *      (fun v => (A) `|` v)     carry a boolean fact A across the call:
 *                               A must only mention variables the call
 *                               does not modify (a frame).          *)
module N = {
  proc twice() : bool = {
    var b;
    b <@ M.coin();
    return b;
  }
}.

(* Scaling: from the 1/2 of coin_e get 1/4 for "res, times 1/2".
 * Write the scaling as  c%xr * v  (an xreal product).  With  c ** v  the
 * concavity side condition sent EasyCrypt r2026.10 into a loop. *)
lemma coin_scaled :
  ehoare [N.twice : (1%r / 4%r)%xr ==> (1%r / 2%r)%xr * res%xr].
proof.
  proc; call /(fun v => (1%r / 2%r)%xr * v) coin_e.
  (* remaining: (1/2) * (1/2) <= 1/4 in xreal; /= computes it *)
  by skip => &hr /=.
qed.

(* Framing: x0 is not modified by the call, so x = x0 can be carried. *)
module P = {
  proc f(x : int) : int = {
    var y;
    y <@ M.incr(x);
    return y + x;
  }
}.

lemma f_K (K : int -> xreal) : ehoare [P.f : K (2 * x + 1) ==> K res].
proof.
  proc.
  exlim x => x0.                                   (* name x *)
  call /(fun v => (x = x0) `|` v) (incr_K (fun r => K (r + x0))).
  (* goal 1: the post of the call (framed) pays for the outer post *)
  + by move=> &hr; apply xle_cxr_r => ->; exact xlexx.
  (* goal 2: the outer pre pays for the pre of the call (framed) *)
  skip => &hr; apply xle_cxr => <- /=.
  by rewrite (_ : x0 + 1 + x0 = 2 * x0 + 1) 1:/# xlexx.
qed.

(* ---------------------------------------------------------------------- *)
(* 5. while: the invariant is an xreal potential.  Pattern:
 *
 *      while ((side facts) `|` potential).
 *      + exit:  !cond /\ I  pays for the post          (move=> &hr; xle_cxr_r)
 *      + body:  ehoare [body : I ==> I]   (with cond in the pre)
 *      the code before the loop must pay for I.
 *
 *    Here: n coins; the expected number of heads is hits + (n - i) / 2. *)
lemma flip_e :
  ehoare [M.flip : (0 <= M.hits) `|` (M.hits%r + 1%r / 2%r)%xr
              ==> (0 <= M.hits) `|` M.hits%xr].
proof.
  proc; seq 1 : ((0 <= M.hits) `|` (M.hits%r + b2r b)%xr).
  + rnd; skip => &hr /=.
    (* First take the side condition out (xle_cxr_r), and only then
     * simplify the expectation: in the other order the goal blows up. *)
```

```
    apply xle_cxr_r ⇒ h; rewrite h /=.
    (* Ep_dbool: 1/2 f true + 1/2 f false; %pos is max(., 0) on reals *)
    by rewrite Ep_dbool /= !to_pos_pos; smt().
  by if; auto ⇒ &hr /=; apply xle_cxr ⇒ -[hb h]; rewrite hb /=; smt().
qed.

lemma run_e :
  ehoare [M.run : (0 ≤ M.hits ∧ 0 ≤ n) `|` (M.hits%r + n%r / 2%r)%xr
             ⟹ M.hits%xr].
proof.
  proc.
  while ((0 ≤ M.hits ∧ 0 ≤ i ≤ n) `|` (M.hits%r + (n - i)%r / 2%r)%xr).
  + (* exit: the potential pays for the post.  /= first merges the nested
       * `|` (cxrA), then xle_cxr_r assumes the facts. *)
    by move⇒ &hr /=; apply xle_cxr_r ⇒ /> *; smt().
  + (* body: fix i and n, frame them across the call of flip, and add the
       * part of the budget the remaining iterations will spend. *)
    wp; exlim i, n ⇒ i0 n0.
    call /(fun v ⇒ (i = i0 ∧ n = n0 ∧ 0 ≤ i0 < n0)
                    `|` (v + ((n0 - (i0 + 1))%r / 2%r)%xr)) flip_e.
    + (* the framed post of flip pays for the invariant after i <- i + 1 *)
      move⇒ &hr /=; apply xle_cxr_r ⇒ /> *.
      apply xle_cxr_l; first smt().
      by rewrite of_realD; smt().
    (* the invariant, with i < n, pays for the framed pre of flip *)
    skip ⇒ &hr /=; apply xle_cxr_r ⇒ /> *.
    smt(to_pos_pos).
  (* before the loop *)
  by auto ⇒ &hr /=; apply xle_cxr ⇒ /> *; smt().
qed.

(* ------------------------------------------------------------------------ *)
(* 6. Expected count → probability: "at least one head" is at most n/2
 *    (Markov-style: the indicator of hits >= 1 is below hits%xr). *)
module R = {
  proc main(n : int) : bool = {
    M.hits <- 0;
    M.run(n);
    return 1 ≤ M.hits;
  }
}.

lemma pr_some_head &m (n0 : int) : 0 ≤ n0 ⇒
  Pr[R.main(n0) @ &m : res] ≤ n0%r / 2%r.
proof.
  move⇒ hn.
  byehoare (: (n = n0) `|` (n0%r / 2%r)%xr ⟹ res%xr) ⇒ //; last by smt().
  proc; wp.
  (* conseq in ehoare: the side goals quantify over what the program
   * modifies, so a fact like "the indicator is below hits" shows up as
   * a forall in a `|` condition. *)
  conseq (: _ ⟹ M.hits%xr).
  + move⇒ &hr; apply xle_cxr_l; last exact xlexx.
    by move⇒ h; case: (1 ≤ h) ⇒ /= ?; smt().
  + by move⇒ &hr; apply xle_cxr_r ⇒ h; exact (h M.hits{hr}).
  call run_e; wp; skip ⇒ &hr /=.
  by apply xle_cxr ⇒ →.
qed.

(* ------------------------------------------------------------------------ *)
(* 7. An abstract adversary: call (: potential).
 *    The adversary makes any number of oracle calls; the oracle answers
 *    at most q of them, and each answered call flips a coin.  Expected
 *    heads ≤ q/2.  Budget the unanswered calls in the potential:
 *        (facts) `|` (hits + (q - n) / 2)
 *    Each answered call spends 1/2 of the budget and pays for the
 *    expected new head; the oracle goal is ehoare [query : I ⟹ I].
 *    (SwapCore.ec: recording_potential, finder_query_preserves_potential.) *)
op q : { int | 0 ≤ q } as ge0_q.

module type Oracle = { proc query() : unit }.

module Ob : Oracle = {
  var n : int
  proc query() : unit = {
```

```
    if (n < q) {
      n <- n + 1;
      M.flip();
    }
  }
}.

module type AdvT (O : Oracle) = { proc run() : unit }.

module H (A : AdvT) = {
  proc main() : unit = {
    M.hits <- 0;
    Ob.n <- 0;
    A(Ob).run();
  }
}.

op pot (hits n : int) : xreal =
  (0 ≤ hits ∧ 0 ≤ n ≤ q) `|` (hits%r + (q - n)%r / 2%r)%xr.

lemma query_pot : ehoare [Ob.query : pot M.hits Ob.n ⟹ pot M.hits Ob.n].
proof.
  proc; if; last first.
  + by skip ⇒ &hr; apply xle_cxr_r ⇒ _; exact xlexx.
  call /(fun v ⇒ (0 ≤ Ob.n ≤ q) `|` (v + ((q - Ob.n)%r / 2%r)%xr)) flip_e.
  + by move⇒ &hr /=; rewrite /pot; apply xle_cxr_r ⇒ /> *; smt(to_pos_pos).
  (* the pre is  (n < q) `|` (pot facts `|` ...) : peel the outer
   * condition first, then relate the two inner ones *)
  wp; skip ⇒ &hr /=; rewrite /pot; apply xle_cxr_r ⇒ /> *.
  by apply xle_cxr ⇒ /> *; smt(to_pos_pos).
qed.

section.

declare module A <: AdvT {-M, -Ob}.

lemma adv_heads : ehoare [H(A).main : (q%r / 2%r)%xr ⟹ M.hits%xr].
proof.
  proc.
  (* weaken the post to the potential; "hits ≤ pot hits n" is the
   * universally quantified side condition *)
  conseq (: _ ⟹ pot M.hits Ob.n).
  + move⇒ _; apply xle_cxr_l; last exact xlexx.
    by move⇒ h n; rewrite /pot; apply xle_cxr_r ⇒ /> *; smt().
  + by move⇒ &hr; apply xle_cxr_r ⇒ h; exact (h M.hits{hr} Ob.n{hr}).
  call (: pot M.hits Ob.n).
  + exact query_pot.
  by auto ⇒ &hr /=; rewrite /pot /=; smt(ge0_q).
qed.

end section.
```

# 10 Samplings

- `rnd f g` couples two draws through a bijection; `rnd{i}` draws on one side only and needs the law to be lossless.
- A draw that the rest of the program does not read can be moved behind a call with `swap`. Drawn last, it is independent of the view, and any event "the sample lies in a set computed from the view" is bounded by the mass of that set.
- The five-argument `seq` splits a phoare bound as $p_1 p_2 + p_3 p_4$.
- `rndsem*{i} n` merges a block of draws into one draw from the composed law, which `rnd` can then relate to a single draw.

$$\frac{f \circ g = \mathrm{id} \text{ on } d_2 \qquad g \circ f = \mathrm{id} \text{ on } d_1 \qquad \forall y \in d_2.\ d_2(y) = d_1(g\,y) \qquad \forall x \in d_1.\ \Psi[x/x_1, f\,x/x_2]}{\models x_1 \xleftarrow{\$} d_1 \sim x_2 \xleftarrow{\$} d_2 : \top \Rightarrow \Psi}\ \texttt{rnd f g}$$

$$\frac{\begin{array}{c}\models c_1 : P \Rightarrow I \qquad \models c_1 : P \Rightarrow R\ [=p_1] \qquad \models c_2 : R \wedge I \Rightarrow Q\ [=p_2] \\ \models c_1 : P \Rightarrow \neg R\ [=p_3] \qquad \models c_2 : \neg R \wedge I \Rightarrow Q\ [=p_4] \qquad p_1p_2 + p_3p_4 = p\end{array}}{\models c_1; c_2 : P \Rightarrow Q\ [=p]}\ \texttt{seq n : R p1 p2 p3 p4 I}$$

When $p_4 = 0$, EasyCrypt does not ask for the premise with $p_3$.

08_sampling.ec

```
(* Samplings: rnd with a bijection, one-sided rnd, moving a draw, product
 * samplings, and the five-argument phoare seq.
 * Run: easycrypt compile 08_sampling.ec *)
require import AllCore Distr DBool DInterval.
require DProd.

(* ---------------------------------------------------------------------- *)
(* 1. rnd f g : couple two draws through a bijection.
 *    rnd f g  relates  x{1} <$ d1  with  x{2} <$ d2  by  x{2} = f x{1},
 *    with g the inverse.  Side goals (all in the skip goal):
 *      forall x2 \in d2, f (g x2) = x2
 *      forall x2 \in d2, mu1 d2 x2 = mu1 d1 (g x2)
 *      forall x1 \in d1, f x1 \in d2 /\ g (f x1) = x1
 *    rnd f alone when f is an involution (f = g).
 *    (SwapGames.ec: rnd (fun y => minv m y) (fun xs => oget m.[xs]);
 *     HRBirthday.ec: rnd (exchange_row a) (exchange_row a).) *)
module Flip = {
  proc pos() : bool = { var b; b <$ {0,1}; return b; }
  proc neg() : bool = { var b; b <$ {0,1}; return !b; }
}.

lemma pos_neg : equiv [Flip.pos ~ Flip.neg : true ==> ={res}].
proof.
  proc; rnd (fun b => !b).
  by skip => />; smt(dbool1E).
qed.

(* An offset on an interval: x{2} = n - 1 - x{1}, an involution of [0..n-1]. *)
module Iv = {
  proc up(n : int) : int = { var x; x <$ [0 .. n - 1]; return x; }
  proc down(n : int) : int = { var x; x <$ [0 .. n - 1]; return n - 1 - x; }
}.

lemma up_down : equiv [Iv.up ~ Iv.down : ={n} ==> ={res}].
proof.
  proc; rnd (fun x => n{1} - 1 - x).
  (* the three side goals, in order: f (g x) = x, equal masses, and
   * f x in the support with g (f x) = x *)
  skip => /> &2; split => [x _ | _]; first by ring.
  split => [x hx | _ x hx]; first by rewrite !dinter1E; smt(supp_dinter).
  smt(supp_dinter).
qed.

(* ---------------------------------------------------------------------- *)
(* 2. rnd{2} : a draw on one side only, never used.  Needs the distribution
 *    to be lossless (side goal  weight d = 1  or  is_lossless d).
 *    Typical for a lazy cache: a repeated query reads the cache on one side
 *    and draws a fresh, discarded sample on the other (SimonFixed.ec). *)
module Waste = {
  proc a() : int = { return 0; }
  proc b() : int = { var x; x <$ {0,1}; return 0; }
}.

lemma waste : equiv [Waste.a ~ Waste.b : true ==> ={res}].
proof. by proc; rnd{2}; skip => />; exact dbool_ll. qed.

(* ---------------------------------------------------------------------- *)
(* 3. Move a draw past a call that does not read it: swap, then the call
 *    with (: true) for an abstract module.  This is how "x is independent of
 *    the adversary's view" becomes provable: draw x AFTER the run, then any
```

```
 *    event "x in S(view)" has probability ≤ mu d (S view)  (LateSample.ec). *)
module type Run = { proc run() : int }.

module Early (R : Run) = {
  proc main() : bool * int = {
    var x, v;
    x <$ {0,1};
    v <@ R.run();
    return (x, v);
  }
}.

module Late (R : Run) = {
  proc main() : bool * int = {
    var x, v;
    v <@ R.run();
    x <$ {0,1};
    return (x, v);
  }
}.

lemma early_late (R <: Run) :
  equiv [Early(R).main ~ Late(R).main : ={glob R} ⟹ ={res, glob R}].
proof. by proc; swap{1} 1 1; rnd; call (: true); auto. qed.

(* ------------------------------------------------------------------------ *)
(* 4. The five-argument seq for phoare: split a probability as a product.
 *
 *      seq n : (R) p1 p2 p3 p4 (I).
 *
 *    For pre ⟹ post with bound b, cutting after n instructions:
 *      goal 1:  hoare  [first n : pre ⟹ I]      (I = true when omitted)
 *      goal 2:  phoare [first n : pre ⟹ R]      = p1   (or ≤ / >=)
 *      goal 3:  phoare [rest : R ∧ I ⟹ post]    = p2
 *      goal 4:  phoare [first n : pre ⟹ !R]     = p3   (dropped if p4 = 0)
 *      goal 5:  phoare [rest : !R ∧ I ⟹ post]   = p4
 *      goal 6:  p1 * p2 + p3 * p4  vs  b
 *    Use 1%r / 0%r for the parts you don't care about; the goals with a
 *    0%r probability are hoare goals in disguise (close with  hoare; ...).
 *    `_` for a probability leaves it as a metavariable (HRRefusal.ec). *)
module Two = {
  proc main() : bool = {
    var b1, b2;
    b1 <$ {0,1};
    b2 <$ {0,1};
    return b1 ∧ b2;
  }
}.

lemma two_quarter : phoare [Two.main : true ⟹ res] = (1%r / 4%r).
proof.
  proc.
  seq 1 : b1 (1%r / 2%r) (1%r / 2%r) (1%r / 2%r) 0%r.
  (* p4 = 0 : EasyCrypt drops the goal for p3 (it is multiplied by 0) *)
  + by auto.                                          (* hoare: to I = true *)
  + by rnd; skip ⇒ />; rewrite dboolE.               (* Pr[b1] = 1/2 *)
  + by rnd; skip ⇒ />; rewrite dboolE.               (* b1 ⟹ Pr[b1 ∧ b2] = 1/2 *)
  + by hoare; rnd; skip ⇒ /> &hr →.                   (* !b1 ⟹ Pr = 0 *)
  by move⇒ _ _; field.                                (* 1/2 * 1/2 + ... *)
qed.

(* ------------------------------------------------------------------------ *)
(* 5. rnd P in a phoare: the probability of the predicate P on the drawn
 *    value.  The skip goal is  mu d P  (relation to the bound)  plus
 *    forall x, (x \in d) ⇒ P x <=> post[x].  Writing P explicitly helps
 *    when the post mentions other variables. *)
lemma coin_true : phoare [Flip.pos : true ⟹ res] = (1%r / 2%r).
proof. by proc; rnd (fun b ⇒ b); skip ⇒ />; rewrite dboolE. qed.

(* ------------------------------------------------------------------------ *)
(* 6. DProd.ProdSampling: one draw from a product  =  two draws.
 *    The library equiv  S.sample ~ S.sample2 ; with  symmetry  for the other
 *    direction, and  conseq  to adapt the pre.  (HRBirthday.ec)
 *    The equiv itself is proved with   rnd : 0 0   (see TACTICS.md). *)
clone DProd.ProdSampling as PS with
```

```
  type t1 <- bool,
  type t2 <- bool.

module Pair = {
  proc one() : bool * bool = {
    var p;
    p <@ PS.S.sample({0,1}, {0,1});
    return p;
  }
  proc two() : bool * bool = {
    var p;
    p <@ PS.S.sample2({0,1}, {0,1});
    return p;
  }
}.

lemma one_two : equiv [Pair.one ~ Pair.two : true ==> ={res}].
proof. by proc; call PS.sample_sample2; auto. qed.

lemma two_one : equiv [Pair.two ~ Pair.one : true ==> ={res}].
proof.
  proc; call (: ={d1, d2} ==> ={res}).
  + by symmetry; conseq PS.sample_sample2 => /#.
  by auto.
qed.

(* ---------------------------------------------------------------------- *)
(* 7. rndsem{i} n : merge the block of draws (and assignments) that starts
 *    after position n into ONE draw from the composed law (dlet / dmap).
 *      rndsem{1} 0   a <$ d1; b <$ d2; c <- (a, b)
 *                    becomes  (a, b, c) <$ dlet d1 (fun a => dmap d2 ...)
 *      rndsem*{1} 0  the same, keeping only what is used later:
 *                    c <$ dlet d1 (fun a => dmap d2 (fun b => (a, b)))
 *    Then one rnd relates it to a single draw on the other side; when the
 *    two laws are equal but not syntactically, give the identity
 *    bijection and prove the masses equal (here with dprod_dlet).
 *    (HRPublic.ec, HRFixed.ec, SimonFixed.ec: rndsem*{1} 0; rnd.) *)
module Sq = {
  proc seq2() : bool * bool = {
    var a, b, c;
    a <$ {0,1};
    b <$ {0,1};
    c <- (a, b);
    return c;
  }
  proc joint() : bool * bool = {
    var p;
    p <$ {0,1} `*` {0,1};
    return p;
  }
}.

lemma seq2_joint : equiv [Sq.seq2 ~ Sq.joint : true ==> ={res}].
proof.
  proc; rndsem*{1} 0.
  rnd (fun p => p) (fun p => p); skip => />.
  by move=> p _; rewrite dprod_dlet.
qed.
```

# 11 Program transformations

- `ecall (H e)` is `call` with an instance of a spec whose arguments are program expressions. With a phoare spec of probability 1 it works one-sided, as `ecall{1}`.
- `rcondt{i} n` and `rcondf{i} n` fix the branch of the `if` or `while` at line `n`. The side goal is a hoare triple on the code before it.
- `while (true) (v)` proves termination with an integer variant.
- `proc*` and then `rewrite equiv[{1} 1 E]` replace a call by the other program of the equiv $E$.

$$\frac{\forall m_2.\ \models c_1 : \Phi(\cdot, m_2) \Rightarrow b \qquad \models c_1; s_1; c_1' \sim c_2 : \Phi \Rightarrow \Psi}{\models c_1; \texttt{if}\ b\ \texttt{then}\ s_1\ \texttt{else}\ s_2; c_1' \sim c_2 : \Phi \Rightarrow \Psi}\ \texttt{rcondt\{1\} n}$$

$$\frac{\forall z.\ \models c : I \wedge b \wedge v = z \Rightarrow I \wedge v < z\ [=1] \qquad I \wedge v \leq 0 \Rightarrow \neg b}{\models \texttt{while}\ b\ \texttt{do}\ c : I \Rightarrow I \wedge \neg b\ [=1]}\ \texttt{while (I) (v)}$$

09_program.ec

```
(* Program-level tactics: ecall, rcondt / rcondf, termination variants,
 * rewrite equiv[...], proc*, sp with two counts.
 * Run: easycrypt compile 09_program.ec *)
require import AllCore Distr DBool.

module M = {
  var c : int

  proc double(x : int) : int = { return 2 * x; }

  proc twice(x : int) : int = {
    var y, z;
    y <@ double(x);
    z <@ double(y);
    return z;
  }

  proc count(n : int) : int = {
    var i;
    i <- 0;
    while (i < n) { i <- i + 1; }
    return i;
  }

  proc guarded(x : int) : int = {
    var r;
    r <- 0;
    if (0 ≤ x) { r <- x; } else { r <- -x; }
    return r;
  }
}.

(* ---------------------------------------------------------------------- *)
(* 1. ecall (lemma args): call with a spec whose pre fixes the arguments.
 *    call needs the pre to match; ecall takes the lemma instance and
 *    substitutes the actual arguments (program expressions are allowed as
 *    lemma arguments).  Works in hoare, phoare, and with {1}/{2} in equiv.
 *    (Simon.ec: ecall{1} (evaluate_exact q{1} m{1} w{1}).)              *)
lemma double_spec (x0 : int) : hoare [M.double : x = x0 ⟹ res = 2 * x0].
proof. by proc; auto. qed.

lemma twice_spec (x0 : int) : hoare [M.twice : x = x0 ⟹ res = 4 * x0].
proof.
  proc.
  ecall (double_spec y).          (* y : the program variable *)
  ecall (double_spec x).
  by auto ⇒ /#.
qed.

lemma double_ll : islossless M.double by proc; auto.

lemma double_exact (x0 : int) :
  phoare [M.double : x = x0 ⟹ res = 2 * x0] = 1%r.
proof. by conseq double_ll (double_spec x0). qed.

(* One-sided ecall in an equiv: a different, exact fact per side. *)
lemma twice_rel : equiv [M.twice ~ M.twice : x{1} = - x{2} ⟹ res{1} = - res{2}].
proof.
  proc.
  ecall{1} (double_exact y{1}); ecall{2} (double_exact y{2}).
  ecall{1} (double_exact x{1}); ecall{2} (double_exact x{2}).
  by auto ⇒ /#.
qed.
```

```
(* ---------------------------------------------------------------------- *)
(* 2. Termination: while (I) (variant).
 *    For islossless / phoare = 1: the variant is an int that decreases at
 *    each iteration and is ≤ 0 when the loop must stop.  Goals: the body
 *    (for every value z of the variant, it decreases) and the code before
 *    the loop, which must establish I and the exit implication.
 *    (MultiSimon.ec: while (true) (q_eval - i); while (true) (size rest).) *)
lemma count_ll : islossless M.count.
proof.
  proc; while (true) (n - i).
  + by move⇒ z; auto ⇒ /#.
  by auto ⇒ /#.
qed.

(* ---------------------------------------------------------------------- *)
(* 3. rcondt / rcondf : decide an if (or the first iteration of a while)
 *    at position n.  Side goal: a hoare triple saying the condition is
 *    true (resp. false) at that point.  rcondt{1} n / rcondf{2} n in an
 *    equiv.  (HRFixed.ec / SimonFixed.ec: rcondf{2} 2; first by auto.) *)
lemma guarded_pos (x0 : int) : hoare [M.guarded : x = x0 ∧ 0 ≤ x0 ⟹ res = x0].
proof.
  proc.
  rcondt 2; first by auto.
  by auto.
qed.

lemma guarded_neg : equiv [M.guarded ~ M.guarded :
                           x{1} = - x{2} ∧ 0 < x{1} ⟹ ={res}].
proof.
  proc.
  (* side goal: forall &m, hoare [prefix : pre, other side read in &m ⟹ cond] *)
  rcondt{1} 2; first by auto ⇒ /#.
  rcondf{2} 2; first by auto ⇒ /#.
  by auto.
qed.

(* A while whose condition is known: rcondt unrolls one iteration. *)
lemma count_one : hoare [M.count : n = 1 ⟹ res = 1].
proof.
  proc.
  rcondt 2; first by auto.
  rcondf 3; first by auto.
  by auto.
qed.

(* ---------------------------------------------------------------------- *)
(* 4. sp n m : symbolic execution of the first n (left) and m (right)
 *    assignments.  sp alone eats as many as possible on both sides; the
 *    counts let you stop before an instruction you want to keep (e.g. one
 *    that writes a variable your invariant reads).  if{1} : case only on
 *    the left if (when the sides branch differently). *)
lemma guarded_left : equiv [M.guarded ~ M.double : x{1} = x{2} ∧ 0 ≤ x{1}
                                                ⟹ 2 * res{1} = res{2}].
proof.
  proc; sp 1 0.
  if{1}.
  + by auto.
  by exfalso; auto ⇒ /#.
qed.

(* ---------------------------------------------------------------------- *)
(* 5. rewrite equiv[{1} n lemma] : replace the call at line n of the left
 *    program by the right program of an equiv lemma (program rewriting).
 *    The lemma must be stated with plain variables (e.g. ={x} ⟹ ={res}).
 *    Use proc* first, so the goal is about "r <@ M.f(args)" itself.
 *    (HRSwap.ec: rewrite equiv[{1} 1 fresh_procedure_exact].)          *)
module Alt = {
  proc twice(x : int) : int = { return 4 * x; }
}.

lemma twice_alt : equiv [M.twice ~ Alt.twice : ={x} ⟹ ={res}].
proof.
  proc; inline *; auto ⇒ /#.
qed.
```

```
lemma twice_sym : equiv [M.twice ~ M.twice : ={x} ==> ={res}].
proof.
  proc*.
  rewrite equiv[{1} 1 twice_alt].
  rewrite equiv[{2} 1 twice_alt].
  by inline *; auto.
qed.
```

# 12 From a random parameter to a fixed one

- A probability $\Pr[G(x_0)@m : res]$ is also a phoare bound for $G$ from every memory that agrees with $m$ on `glob G`. In that form it can be a step of a bigger program.
- If $G$ succeeds with probability at least $\varepsilon$ on every value of the support, it does on a random value. On a finite support, one value does no better than the average.
- `case @[ambient]: (e)` splits on `e` in the ambient logic, inside a program goal.

10_averaging.ec

```
(* From a random parameter to a fixed one: a game's probability as a phoare,
 * averaging, and "some value is no worse than the average".
 * Run: easycrypt compile 10_averaging.ec
 * (Condensed from simon/src/util/Averaging.ec.) *)
require import AllCore Distr List StdOrder.
import RealOrder.

type input.
op d : input distr.

module type Game = { proc main(x : input) : bool }.

module Avg (G : Game) = {
  proc main() : bool = {
    var x, b;
    x <$ d;
    b <@ G.main(x);
    return b;
  }
}.

(* ------------------------------------------------------------------------ *)
(* 1. A Pr[...] as a phoare bound, to be used as a step of a larger program.
 *    Fix the module's state to the one of &m:  (glob G) = (glob G){m}.
 *    bypr turns the phoare into a statement about Pr from every memory &m0
 *    satisfying the pre; byequiv + proc true (= call with the trivial
 *    invariant for an abstract procedure) relates it to &m. *)
lemma game_ph (G <: Game) (x0 : input) &m :
  phoare [G.main : arg = x0 /\ (glob G) = (glob G){m} ==> res]
    = (Pr[G.main(x0) @ &m : res]).
proof.
  (* bypr on a phoare "= p" also asks for 0 <= p *)
  bypr => &m0; split => [|[->> hG]]; first smt(mu_bounded).
  by byequiv (: ={arg, glob G} ==> ={res}) => //; proc true; auto.
qed.

(* ------------------------------------------------------------------------ *)
(* 2. Averaging from below.
 *    - byphoare (: (glob G) = (glob G){m} ==> res)  keeps the link to &m.
 *    - seq 1 with the 5-argument form (08_sampling.ec, 4.).
 *    - exlim x => xx  then  case @[ambient]: (xx \in d)  : the case
 *      analysis is done in the AMBIENT logic (not on the program state),
 *      so each branch has the plain fact  xx \in d  as a hypothesis.
 *    - conseq (: _ ==> _ : = p) : change the bound of a phoare goal to
 *      the one of the lemma; the side goal compares the two bounds. *)
lemma avg_ge (G <: Game) (eps : real) &m :
  is_lossless d =>
  (forall x, x \in d => eps <= Pr[G.main(x) @ &m : res]) =>
  eps <= Pr[Avg(G).main() @ &m : res].
proof.
  move=> hll h; case: (eps <= 0%r) => heps; first smt(mu_bounded).
  byphoare (: (glob G) = (glob G){m} ==> res) => //; proc.
```

```
  seq 1 : (x \in d ∧ (glob G) = (glob G){m}) 1%r eps 0%r 0%r ⇒ //.
  + by rnd; skip ⇒ />; rewrite -weightE_support hll.
  exlim x ⇒ xx; case @[ambient]: (xx \in d) ⇒ hxx; last by exfalso; smt().
  call (: arg = xx ∧ (glob G) = (glob G){m} ⟹ res); last by skip ⇒ />.
  conseq (: _ ⟹ _ : = (Pr[G.main(xx) @ &m : res])).
  + by smt().
  exact (game_ph G xx &m).
qed.

(* ---------------------------------------------------------------------- *)
(* 3. A value no better than the average, on a finite support s.
 *    The minimiser over a list (argmin_list) plus averaging from below. *)
lemma argmin_list ['a] (f : 'a → real) (s : 'a list) :
  s ≠ [] ⇒ exists x, x \in s ∧ forall y, y \in s ⇒ f x ≤ f y.
proof.
  elim: s ⇒ // x s ih _; case: (s = []) ⇒ [→|hs]; first by exists x.
  smt().
qed.

lemma exists_le_avg (G <: Game) (s : input list) &m :
  is_lossless d ⇒ s ≠ [] ⇒ (forall x, x \in d <=> x \in s) ⇒
  exists x, x \in d ∧ Pr[G.main(x) @ &m : res] ≤ Pr[Avg(G).main() @ &m : res].
proof.
  move⇒ hll hs hd.
  have /= [x0 [hin hmin]] := argmin_list (fun x ⇒ Pr[G.main(x) @ &m : res]) s hs.
  exists x0; split; first by rewrite hd.
  by apply (avg_ge G (Pr[G.main(x0) @ &m : res]) &m hll) ⇒ x hx; apply hmin; rewrite -hd.
qed.
```

# 13 Pitfalls

- `forall` extends as far to the right as it can. In an invariant write `∧ (forall y, P y) ∧ Q` with the parentheses, or `Q` ends up under the binder.
- `split` and other tactic names cannot be used as lemma or hypothesis names.
- Program expressions cannot mention `glob M`. EasyCrypt reports that expressions cannot contain a glob statement.
- Inside a section, a `declare axiom` cannot use a `local` predicate, and a non-local operator cannot mention the `glob` of a declared module. Spell the predicate out in the axiom.
- `⇒ //` may close more goals than expected, and the next bullet then fails because all goals are closed.
- `ecall{1} (lemma e{1})` crashed with `InvalidGoalShape` in an earlier version. In `r2026.10` it works with phoare specs of probability 1 (Section 11); `seq`, `exlim` and `call{1}` remain an alternative.
- In `ehoare`, scale with `call /(fun v ⇒ p%xr * v)`, not `p ** v`: the latter made the concavity check loop.
- The order of the side goals from `conseq` is not always the order of its arguments. Look at them before writing bullets.

# 14 Seeing the goals from the command line

`easycrypt llm` loads a file up to a line and prints the goals there. In release `r2026.10` the options are `-upto LINE` (the goals before that line; `LINE:COL` also works) and `-lastgoals` (the goals at the first failure):

```
easycrypt llm -I <dirs> -upto 42 file.ec
easycrypt llm -I <dirs> -lastgoals file.ec
```

Other builds also accept an interactive `-eval` script:

```
easycrypt llm -I <dirs> -eval 'LOAD "file.ec" 42
QUIET ON
proc.
GOALS ALL'
```

- `LOAD` checks everything up to and including the given line, then waits for more input. If that line is `proof. admit. qed.`, split it first, or the load runs past the proof.
- Every further line is one tactic, one sentence per line.

- `GOALS` prints the focused goal, `GOALS ALL` prints all of them, and `TREE ALL` prints the goal tree. An error prints `ERROR` together with the offending `source:` line.